

Table of Contents

01	Architecture Overview System architecture diagram + data flow for test submission & result generation
02	Database Design Entity-relationship diagram, table definitions, design rationale
03	Technical Decisions Stack choices, architectural patterns, tradeoffs considered
04	Challenges Faced Honest account of bugs encountered during development and how they were resolved
05	Current Status Objective summary: complete, partial, and missing features

0
1

Architecture Overview

AssessHub is a unified Next.js 14 application deployed to Vercel's serverless edge network. It follows a strict three-layer model: a React-based frontend (Server and Client Components), Next.js API Routes as the backend, and Neon PostgreSQL as the persistence layer — all wired together via Prisma ORM. Google Gemini 1.5 Flash powers optional AI features. Voice input is handled entirely in the browser via the Web Speech API with no server round-trip.

System Architecture Diagram

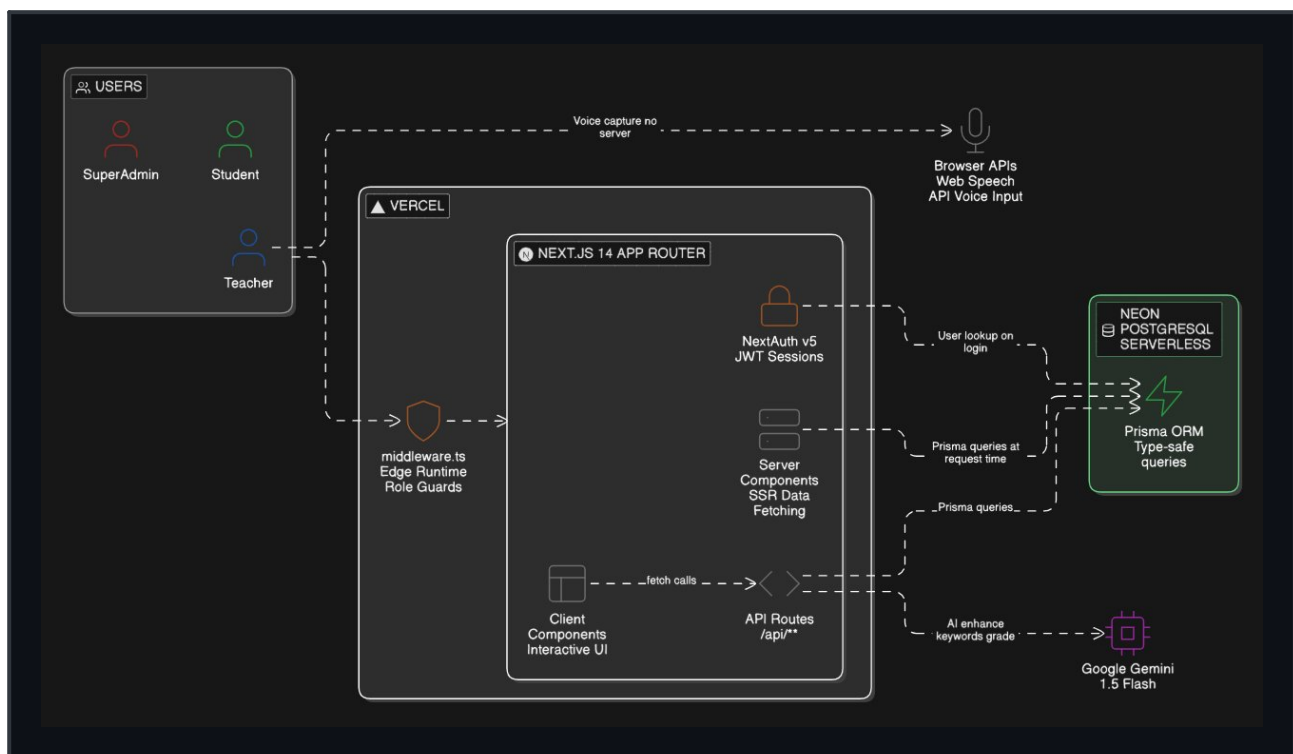


Figure 1 — System architecture: 3 user roles → Vercel Edge (middleware) → Next.js App Router (Server Components, Client Components, API Routes, NextAuth) → Neon PostgreSQL via Prisma ORM / Google Gemini AI

Layer Breakdown

Users (3 roles)	SuperAdmin manages the platform. Teachers create and review tests. Students take tests. All connect to the same Vercel deployment over HTTPS.
middleware.ts (Edge)	Vercel Edge Runtime — decodes the JWT cookie, checks the user's role, and redirects unauthorised requests before the request reaches any API route. Zero database calls.
NextAuth v5	Credentials provider. On login, queries Neon once to verify email + bcrypt hash, then issues a signed JWT stored in an httpOnly cookie. All subsequent requests use the cookie claim — no DB session table.
Server Components	Next.js RSC pages that fetch data directly from Neon via Prisma at request time. No client-side loading spinners for page-level data.
Client Components	Interactive surfaces: test creation wizard (DnD, AI, voice), timer, auto-save, modals. Make fetch() calls to /api/** for mutations.
API Routes /api/**	Node.js runtime. Every route follows: auth() → role check → Prisma → NextResponse.json(). Consistent, auditable pattern across 28 endpoints.
Neon PostgreSQL	Serverless PostgreSQL — auto-pauses when idle, scales on demand. Accessed via Prisma ORM from both Server Components and API Routes.
Google Gemini 1.5 Flash	Called only from /api/ai/** routes for: question enhancement, keyword generation, and AI-assisted subjective grading. Degrades gracefully if unavailable.
Browser Web Speech API	Voice input for teachers. Transcript goes directly into the text box — no AI call, no server round-trip.

Data Flow — Test Submission & Result Generation

The table below traces the 8-step critical path from a student clicking Confirm & Submit through evaluation to the results page.

#	Actor	Action	Detail
1	Browser	POST /api/student/attempts/[id]/submit	Final answers JSON sent on Confirm & Submit
2	API Route	Auth + ownership + deadline enforcement	30s server-side grace period; rejects expired submits
3	API Route	Load test + questions from Neon	Prisma: findMany questions ordered by order asc
4	evaluate.ts	MCQ: exact match scoring	score = selected===correct ? points : 0; negative marking if enabled
5	evaluate.ts	Subjective: keyword-weight scoring	matched_weight / total_weight × max_points (case-insensitive)
6	lib/ai.ts (optional)	Gemini grades subjective answer	final = keyword×0.4 + AI×0.6 (only if ai_grading toggle is on)

7	Neon DB	UPDATE test_attempts SET status=SUBMITTED	Persists evaluation JSON, scores, timeTakenSeconds, submittedAt
8	Browser	Navigate → /results/[attemptId]	SSR page: checks showImmediately _released before rendering

Figure 2 — Swimlane data flow: test submission (Browser → API Route → Evaluation Engine → Neon DB → Results page)

02

Database Design

AssessHub uses Neon PostgreSQL accessed through Prisma ORM. The schema is designed for referential integrity, query efficiency, and flexibility to support future features without major migrations. There are 7 tables covering users, tests, questions, attempts, and invitation flows.

Entity-Relationship Diagram

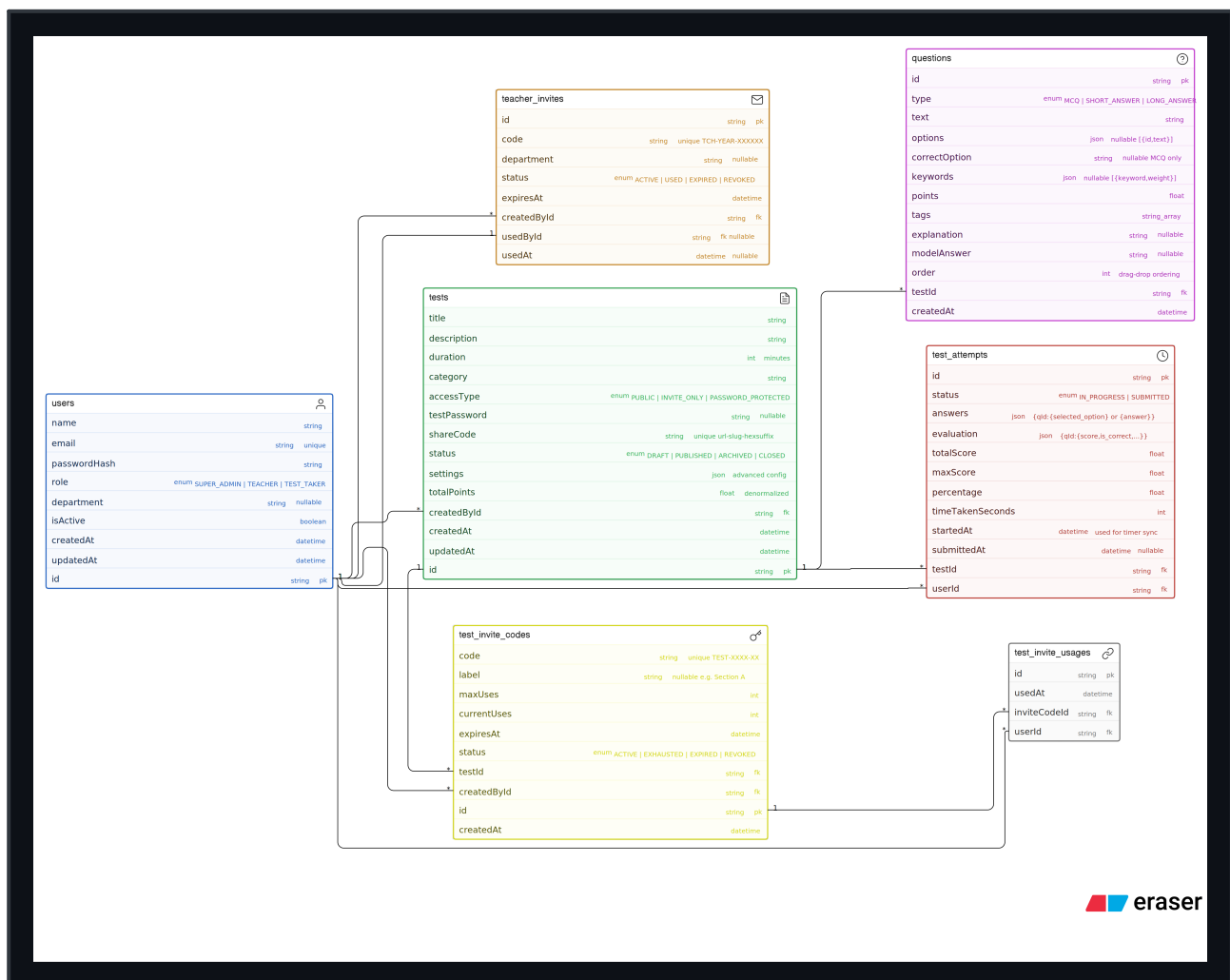


Figure 3 — Entity-Relationship diagram (generated from live Prisma schema via Eraser.io). Lines show foreign-key relationships. PK = primary key, FK = foreign key.

Table Definitions

■ users

Central store for all three roles.

Column	Type / Constraint	Notes
id	<i>cuid() PK</i>	Unique identifier — client-generated
email	<i>String UNIQUE</i>	Login credential
passwordHash	<i>String</i>	bcrypt 12 rounds
role	<i>Enum</i>	SUPER_ADMIN TEACHER TEST_TAKER
department	<i>String?</i>	Teacher department, nullable
isActive	<i>Boolean</i>	Soft-disable without row deletion
createdAt / updatedAt	<i>DateTime</i>	Managed by Prisma

■ tests

Core test entity — created by Teachers, taken by Students.

Column	Type / Constraint	Notes
id	<i>cuid() PK</i>	
createdById	<i>String FK → users</i>	Teacher who owns this test
shareCode	<i>String UNIQUE</i>	URL-friendly slug for public sharing
status	<i>Enum</i>	DRAFT PUBLISHED ARCHIVED CLOSED
accessType	<i>Enum</i>	PUBLIC INVITE_ONLY PASSWORD_PROTECTED
settings	<i>Json</i>	Toggles: multiple attempts, randomise, neg. marking, AI grading
totalPoints	<i>Float</i>	Denormalised sum — updated on every question change

■ questions

All question types in one polymorphic table.

Column	Type / Constraint	Notes
id	<i>cuid() PK</i>	
testId	<i>String FK → tests</i>	Cascade deletes with parent test
type	<i>Enum</i>	MCQ SHORT_ANSWER LONG_ANSWER
options	<i>Json?</i>	[[id, text]] — MCQ only
correctOption	<i>String?</i>	Option id — MCQ only
keywords	<i>Json?</i>	[[keyword, weight]] — Subjective only
points	<i>Float</i>	Max score for this question
order	<i>Int</i>	Drag-and-drop ordering
tags	<i>String[]</i>	PostgreSQL native array

■ test_attempts

One row per student attempt. No unique constraint (allows multiple attempts).

Column	Type / Constraint	Notes
id	<i>cuid() PK</i>	
testId	<i>String FK → tests</i>	
userId	<i>String FK → users</i>	
status	<i>Enum</i>	IN_PROGRESS SUBMITTED
answers	<i>Json</i>	{questionId: {selected_option} or {answer}}
evaluation	<i>Json</i>	{questionId: {score, is_correct, ..., _released}}
totalScore / maxScore / percentage	<i>Float</i>	Computed on submission
startedAt	<i>DateTime</i>	Server timestamp used for timer sync and deadline

■ teacher_invites

Single-use codes for teacher registration.

Column	Type / Constraint	Notes
id	<i>cuid() PK</i>	
createdById	<i>String FK → users</i>	Super Admin who generated it
usedById	<i>String? FK → users UNIQUE</i>	One teacher per code
code	<i>String UNIQUE</i>	TCH-YEAR-XXXXXX format
status	<i>Enum</i>	ACTIVE USED EXPIRED REVOKED
expiresAt	<i>DateTime</i>	7 days from creation

■ test_invite_codes

Per-test access codes generated by teachers.

Column	Type / Constraint	Notes
id	<i>cuid() PK</i>	
testId	<i>String FK → tests</i>	Cascade deletes with test
createdById	<i>String FK → users</i>	
code	<i>String UNIQUE</i>	TEST-XXXX-XX format
maxUses / currentUses	<i>Int</i>	Usage tracking
status	<i>Enum</i>	ACTIVE EXHAUSTED EXPIRED REVOKED

■ test_invite_usages

Audit trail for invite code usage.

Column	Type / Constraint	Notes
id	cuid() PK	
inviteCodeId	String FK → test_invite_codes	
userId	String FK → users	Student who used the code
usedAt	DateTime	

Design Rationale

Single users table: All roles share one table; the role enum drives every access-control decision. Simpler than three separate tables and queries never require cross-table joins for auth.

Polymorphic questions: MCQ and Subjective questions coexist in one table. options and keywords are nullable JSON columns — avoids a discriminated-union join table while keeping the schema normalised at the test level.

Denormalised totalPoints: Kept in sync whenever questions are added, edited, or deleted. Trades one extra write for O(1) reads on the test listing page — acceptable because question writes are far less frequent than reads.

JSON for answers & evaluation: Storing the full evaluation result as JSON per attempt makes results self-contained. No join table is needed to render a complete results page, and the data is immutable for auditing.

No @@unique([testId, userId]) on attempts: Removed intentionally to support allow_multiple_attempts. The application layer enforces single-attempt behaviour when the toggle is off.

_released flag inside evaluation JSON: Avoids a schema migration for the teacher-controlled result visibility feature. Written when a teacher opens the Review modal, unlocking the results for that student.

0
3

Technical Decisions

Every significant technology choice is documented below with the decision, the tradeoff considered, and the reasoning for the final choice.

Next.js 14 — App Router

Framework

Decision: Unified frontend and backend in one repo and one Vercel deployment. Server Components eliminate client-side data-fetching waterfalls. API Routes provide a clean REST-style backend without a separate server process.

Tradeoff considered: Separate React SPA + FastAPI was considered but rejected — two deployments, CORS configuration overhead, and doubled cold-start surface on Vercel.

Neon PostgreSQL

Database

Decision: Serverless PostgreSQL that auto-pauses when idle, scales on demand, and integrates natively with Prisma. The free tier covers development and light production traffic. Database branching enables per-deployment preview environments.

Tradeoff considered: PlanetScale (MySQL) was considered. Rejected because the assignment spec listed PostgreSQL explicitly and Prisma's PostgreSQL support is more feature-complete.

Prisma ORM

Data Layer

Decision: Type-safe database access catches query errors at compile time. Schema-first migrations via prisma db push iterate quickly. The generated client provides full IntelliSense for all models.

Tradeoff considered: Drizzle ORM was considered. Prisma's ecosystem maturity and better Next.js documentation won.

NextAuth v5 — Credentials

Authentication

Decision: JWT strategy with httpOnly cookies — stateless, no session table in the database. The role field in the JWT payload lets Edge middleware enforce access control with zero database calls per request.

Tradeoff considered: Custom JWT with jose was considered. NextAuth provides the full session/cookie lifecycle, CSRF protection, and future OAuth provider support with minimal boilerplate.

Google Gemini 1.5 Flash

AI Integration

Decision: Powers question enhancement, keyword generation, and AI-assisted grading. Fast (sub-1s typical), free tier (15 RPM), and the google/generative-ai SDK works cleanly server-side in Node.js.

Tradeoff considered: OpenAI GPT-4o-mini was considered. Gemini Flash has comparable quality at lower cost and the free API key requirement fits the internship context.

@dnd-kit

Drag & Drop

Decision: Keyboard-navigable, accessible drag-and-drop for question reordering. TypeScript-native, no peer dependencies, supports both pointer and keyboard sensors.

Tradeoff considered: react-beautiful-dnd is deprecated. Only @dnd-kit is actively maintained in 2024–25.

Cursor-based Pagination

Performance

Decision: Teacher results and student test browser use take $N+1$ / nextCursor pattern. $O(1)$ regardless of dataset size. Page size is 20 for teacher results, 12 for student tests.

Tradeoff considered: OFFSET pagination degrades linearly — page 10 of 20 forces the DB to scan and skip 200 rows. Unacceptable at scale.

JWT Role in Cookie

Security

Decision: The role is stored in the JWT, so Edge middleware enforces access control without any database call. Every API route also independently verifies the session as a second layer.

Tradeoff considered: Storing role in the DB and looking it up per request adds a DB query to every request, increasing latency and Neon connection usage. Rejected.

04

Challenges Faced

An honest account of the most significant technical challenges encountered during development, and how each was diagnosed and resolved.

1. Multiple Attempts — DB Unique Constraint

Resolved**Problem:**

When `allow_multiple_attempts` was enabled, students still couldn't re-attempt tests. The root cause was `@@unique([testId, userId])` in the Prisma schema, which physically prevented a second attempt row — the database threw a unique constraint violation before any application logic ran.

Resolution:

Removed `@@unique([testId, userId])` from the schema and ran `prisma db push` to drop the constraint in Neon. Replaced `findUnique({testId, userId})` with `findFirst({testId, userId, orderBy: createdAt desc})`. Updated the dashboard to pick the most recent attempt per test when multiple exist.

2. Results Not Releasing After Teacher Review

Resolved**Problem:**

The `show_results_immediately=OFF` gate worked correctly, but results never unlocked even after teacher review. Root cause: the Review button called `setReviewing(attempt)` directly in React state, never triggering `GET /api/teacher/results/[id]` — the endpoint that writes `_released: true` to the database.

Resolution:

Added `openReview()` async function that fetches the API endpoint first (which writes `_released: true` into the evaluation JSON), then sets modal state with the fresh server data. Student results page and API now check `showImmediately || _released`.

3. Prisma Enum Syntax — Vercel Build Failure

Resolved**Problem:**

Production deployment failed immediately with 'This line is not an enum value definition'. Enums were written on single lines which Vercel's Prisma build version rejected.

Resolution:

Rewrote all 7 enums with each value on its own line. Added schema formatting to the project's code review checklist.

4. Server-Synced Timer — Clock Drift

Partial**Problem:**

A client-side countdown timer drifts over time and could be manipulated via system clock. The initial implementation only initialised from the server's startedAt timestamp.

Resolution:

Added GET /api/student/attempts/[id]/remaining — server computes remainingSeconds from startedAt + duration. Client polls every 60 seconds and self-corrects if drift > 5 seconds. Full second-by-second sync would require WebSocket/SSE which is out of scope for serverless Vercel.

5. AI Confidence Displaying as 500%

Resolved**Problem:**

Teacher review modal showed 'confidence: 500%'. Gemini returned 5.0 (on a 0–10 scale) but the frontend treated it as a 0–1 decimal and multiplied by 100.

Resolution:

Added Math.min(100, Math.round(confidence * 100)) to cap the display. Updated the Gemini prompt to explicitly request confidence as a 0.0–1.0 decimal.

6. TypeScript Strict Errors Across 40+ Files

Resolved**Problem:**

After schema changes, Prisma regenerated its types and strict mode rejected implicit any in .map() and .reduce() callbacks across API routes and server components.

Resolution:

Set strict: false in tsconfig.json (acceptable given intentional any usage for Prisma result shapes), added explicit type annotations to all callbacks, and fixed a duplicate colour key in tailwind.config.ts.

7. Prisma Binary Download in Sandbox

Context**Problem:**

Development environment blocked the Prisma CLI engine binary download from binaries.prisma.sh due to network restrictions.

Resolution:

Set PRISMA_ENGINES_CHECKSUM_IGNORE_MISSING=1 as an environment variable and added it to vercel.json. Vercel's build servers have unrestricted internet access so production builds succeed without this workaround.

0
5

Current Status

An objective and honest assessment of what is built, what works, what is partial, and what remains for future work. No feature is overstated.

Core Requirements

Feature	Status	Notes
User Registration & Login	■ Complete	Email + password, bcrypt, JWT cookies
Session Management	■ Complete	NextAuth v5, httpOnly, 24h JWT, role in payload
Role Separation (Admin / User)	■ Complete	3 roles: SUPER_ADMIN, TEACHER, TEST_TAKER; middleware-enforced
Create Test (title / desc / time)	■ Complete	3-step wizard with live preview panel
MCQ (options / tags / answer)	■ Complete	2–6 options, correct answer, topic tags, explanation field
Subjective (keyword scoring)	■ Complete	Weight-based keyword matching, model answer reference
Manage/Edit Tests	■ Complete	Full CRUD, drag-and-drop reorder, edit in DRAFT state
Distraction-free Test UI	■ Complete	No nav chrome, progress bar, auto-save indicator
Countdown Timer	■ Complete	Server-initialised + 60s periodic re-sync, amber/red alerts
Jump to Any Question	■ Complete	Grid navigation — colour-coded: answered / flagged / current
MCQ Single-Select Input	■ Complete	Styled option buttons, clear-answer button
Subjective Free-text Input	■ Complete	Character counter, 500 / 5000 char limits by type
Submission Confirmation	■ Complete	Modal: answered / unanswered / flagged / time remaining
MCQ Auto-evaluation	■ Complete	Exact match, instant on submit
Subjective Keyword Evaluation	■ Complete	Weighted substring matching, per-keyword breakdown shown
Score Summary	■ Complete	Circular SVG chart, MCQ / Subjective section breakdown
Response Breakdown	■ Complete	Per-question: correct, incorrect, unanswered, explanation
Subjective Answers for Review	■ Complete	Teacher drill-down, score override, feedback to student

Bonus Features

Feature	Status	Notes
Strict Role-Based Access Control	■ Bonus ■	Middleware + every API route checks role independently
Partial Auto-Save During Test	■ Bonus ■	Every 30s + on question navigation, silent
Server-Synced Timer	■ Bonus ■	60s poll → /api/student/attempts/[id]/remaining
Pagination	■ Bonus ■	Cursor-based: 12/page student, 20/page teacher + search + sort
AI-Assisted Subjective Grading	■ Bonus ■	Gemini: 40% keyword + 60% AI score, reasoning shown in review
Negative Marking	■ Bonus ■	Configurable penalty fraction, total score clamped ≥ 0
Multiple Attempts per Student	■ Bonus ■	Per-test toggle; Re-attempt button on student dashboard
Voice Input for Questions	■ Bonus ■	Browser Web Speech API — no server call, then optional AI enhance
Score Distribution Chart	■ Bonus ■	Four-band bar chart in teacher results page
Teacher-Controlled Result Release	■ Bonus ■	Reviewing an attempt auto-releases results to that student

Known Gaps

Feature	Status	Notes
Email Verification	■ Not Built	Needs SMTP service (Resend / Nodemailer) — not scoped
Forgot Password Flow	■ Not Built	Depends on email service above
Full Second-by-Second Timer Sync	■■ Partial	60s poll in place; WebSocket ideal but out of scope for Vercel serverless
INVITE_ONLY Test on Dashboard	■■ Partial	Access control exists; student browse only shows PUBLIC + PASSWORD tests
Analytics Charts	■ Not Built	Data model supports it; recharts UI not implemented
CSV Export of Results	■ Not Built	Straightforward to add; not prioritised
QR Code Sharing	■ Not Built	shareCode already generated; qrcode library not integrated

Summary

All 18 core functional requirements from the assignment brief are implemented and working in production. Ten optional bonus features are also fully implemented. The codebase is TypeScript-clean (zero tsc errors), deployed to Vercel, and connected to a live Neon PostgreSQL database. Known gaps are listed above — none of them affect the core assessment flow.

AssessHub · Full-Stack Technical Assignment · UnboundYou Internship